Floating point processing vs. visualization

processing representation ≠ display representation

2D / grayscale image

cv2.imread() — uint8 [0,255]

↓

astype(np.float64) — float64 [0.0,255.0]

↓

Sobel / convolution — float64 (arbitrary range)

↓

plt.imshow(..., cmap='gray') — Matplotlib maps values to the colormap

The underlying NumPy array remains float64 and its values are unchanged.

3D / RGB

RGB display conventions

uint8 → values in [0,255]
float → values in [0,1]

OpenCV output type

filter2D / sepFilter2D with ddepth = -1 preserve the input depth.
float64 in → float64 out; uint8 in → uint8 out.

Why process derivatives in float?

Derivative responses can be negative or exceed 255.

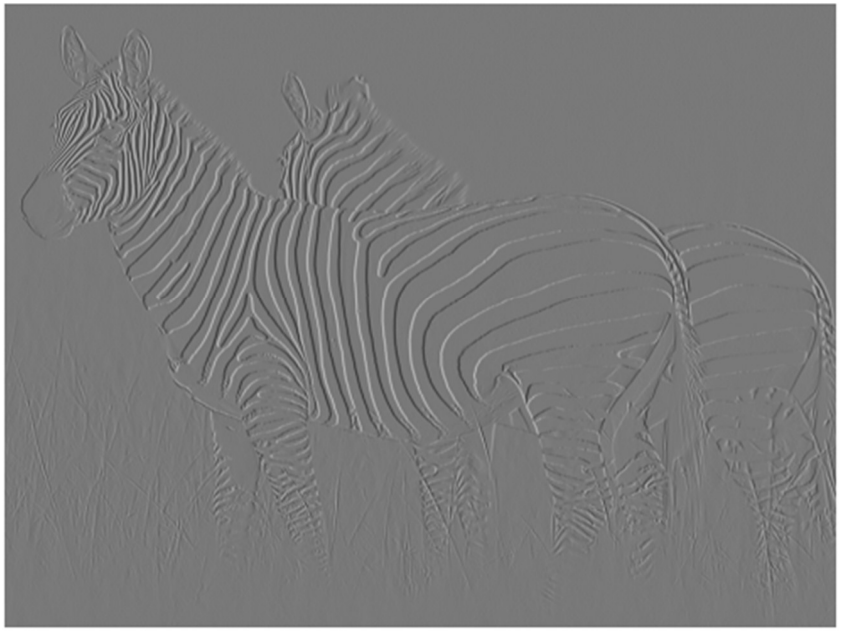
A uint8 output cannot represent negative values or values above 255; OpenCV saturates them to the valid range [0,255].

imshow renders the data; it does not convert your Python array to uint8.

Floating point processing vs. visualization

2D / grayscale image

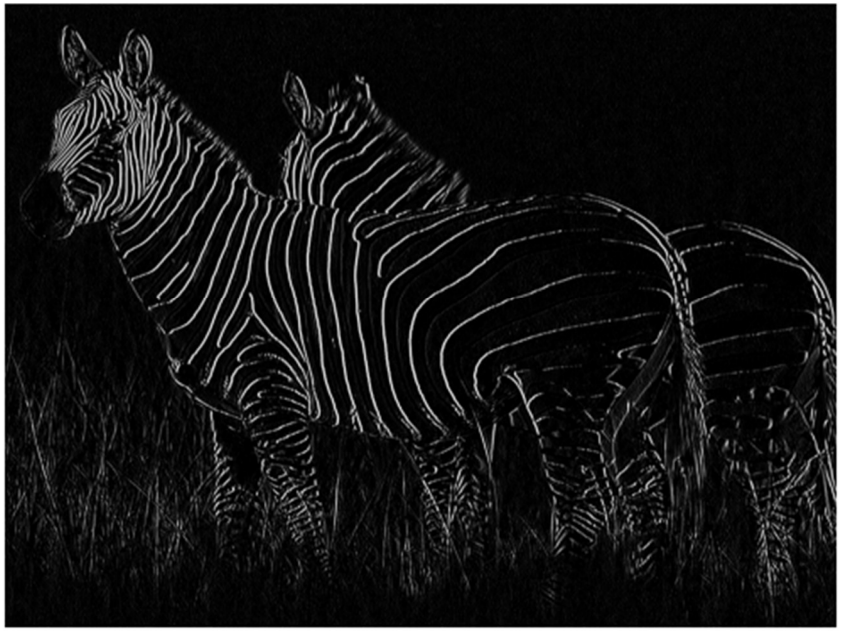
cv2.Sobel(im_float,ddepth=-1,dx=1,dy=0)



float64 -779.0 856.0

For a 2D grayscale image, Matplotlib automatically **normalizes the displayed values between** vmin and vmax (by default, **the data minimum and maximum**):
-779 → 0
856 → 1

cv2.Sobel(im_uint8,ddepth=-1,dx=1,dy=0)



uint8 0 255

uint8 output **clips/saturates responses to [0,255]**: negative values become 0 and values above 255 become 255.

Floating point processing vs. visualization

float en grayscale (2D) ≠ float en RGB (3D)

3D / RGB

RGB float64 in [0,255]



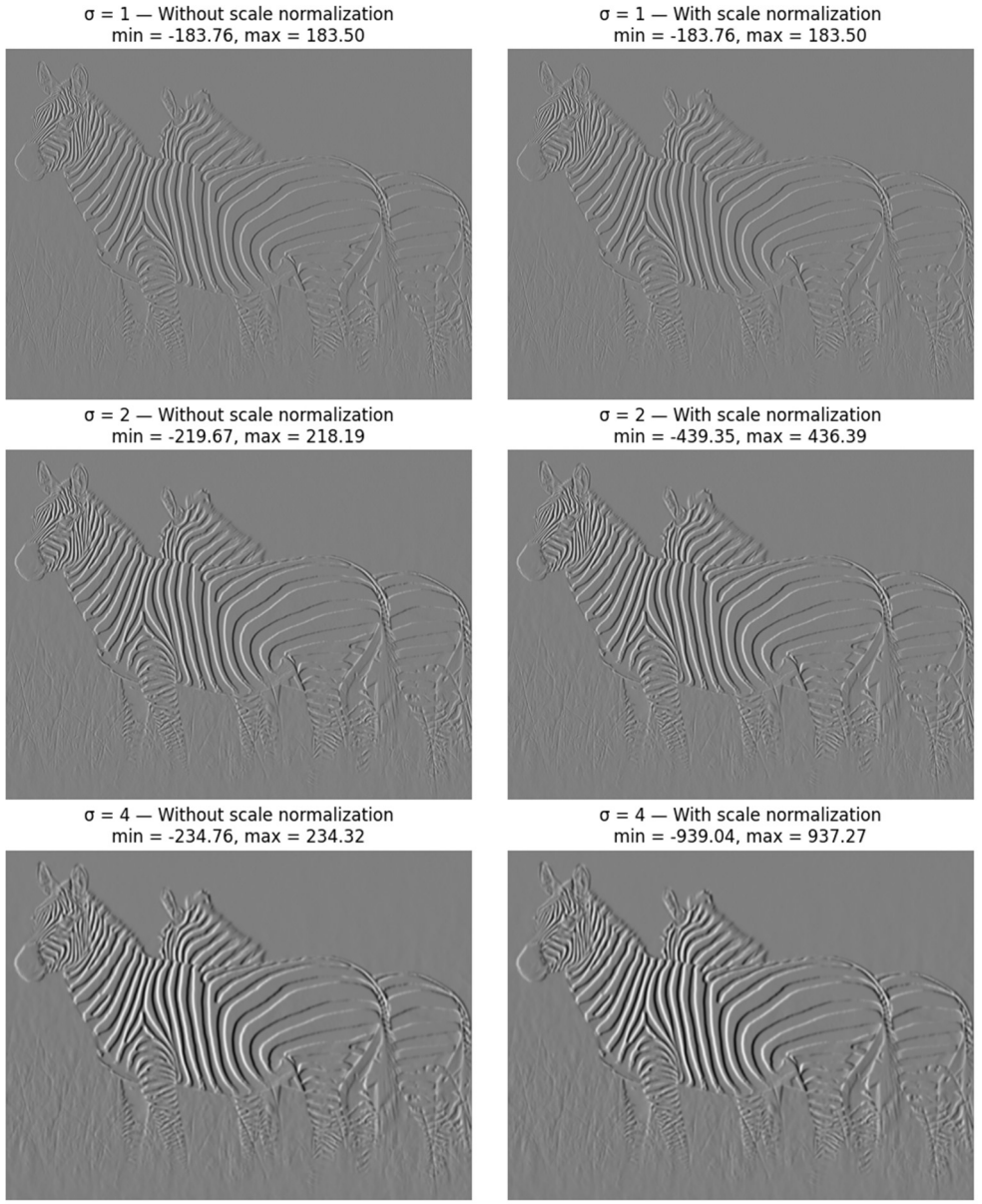
RGB float64 in [0,1]



For RGB float images, values must lie in [0,1].
This affects visualization only; the NumPy
array itself is not modified.

Processing vs. Visualization

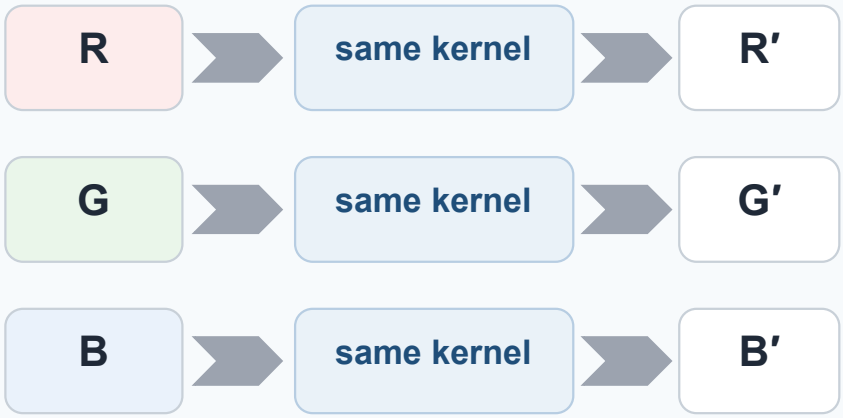
- Visualizing Gaussian derivatives with and without scale normalization.
  - The input image is grayscale. Matplotlib will automatically scale the grayscale values, so no differences are visible (although the matrix values are different).



What happens when we filter a color image?

- OpenCV does not automatically convert RGB/BGR images to grayscale. It is **very common to directly operate in grayscale**.

Direct filtering of a color image



GaussianBlur, filter2D and Sobel process each channel independently.

But what is “the” color gradient?

Sobel on RGB gives: GxR, GxG, GxB
GyR, GyG, GyB

Common classical choice

Convert to grayscale first → one Gx, one Gy, one magnitude and one orientation.

Smoothing: usually keep color.
Edge detection: grayscale is often the simplest classical option.

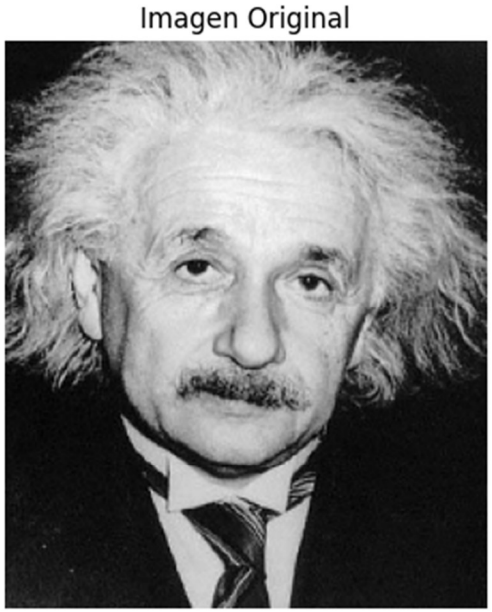
Why is a kernel called “kernel”?

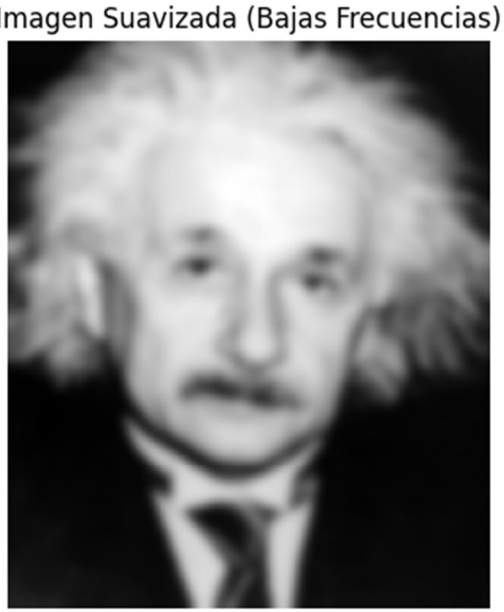
It is called a kernel because **it lies at the core of the filtering operation**: its coefficients determine how the neighborhood of each pixel is combined.

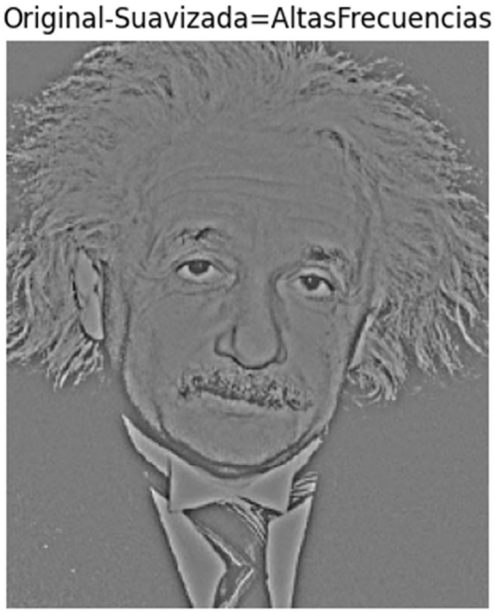
The word kernel means “core” or “central part”.

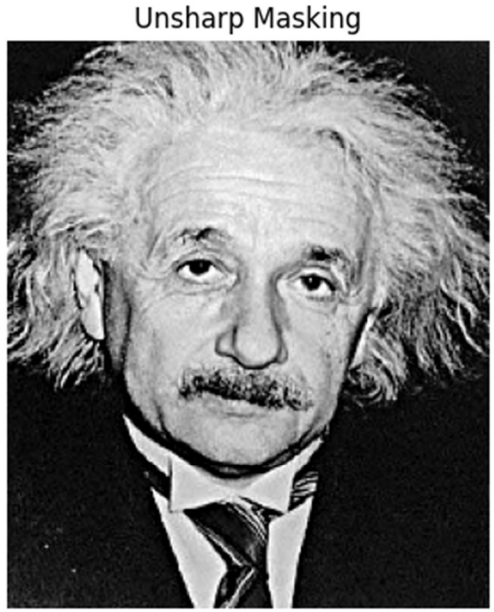
This use of kernel is different from the algebraic notion of the kernel of a linear transformation.

What happens if we recursively apply *unsharp masking*?









$$F + \alpha\left(F - F * H\right)$$

$$\alpha = 1.0$$

What happens if we recursively apply *unsharp masking*?

Imagen Original

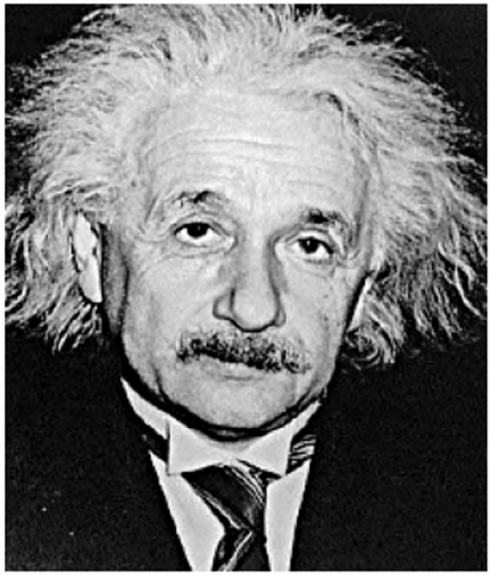
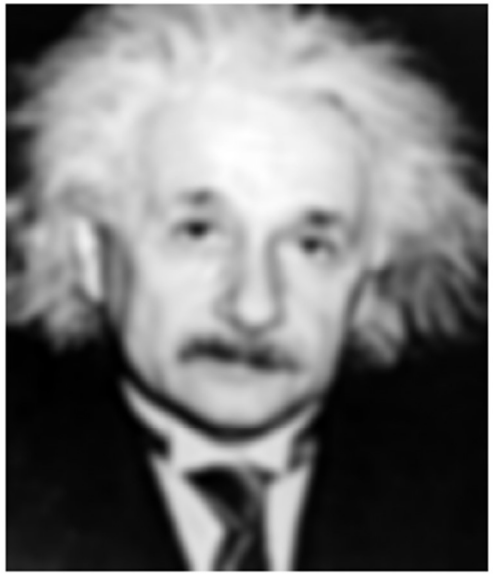
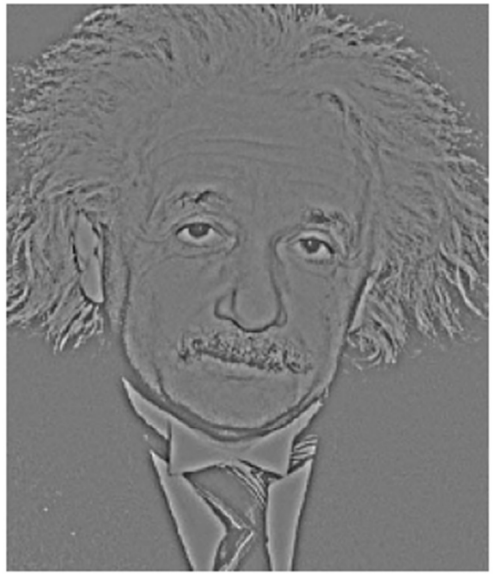


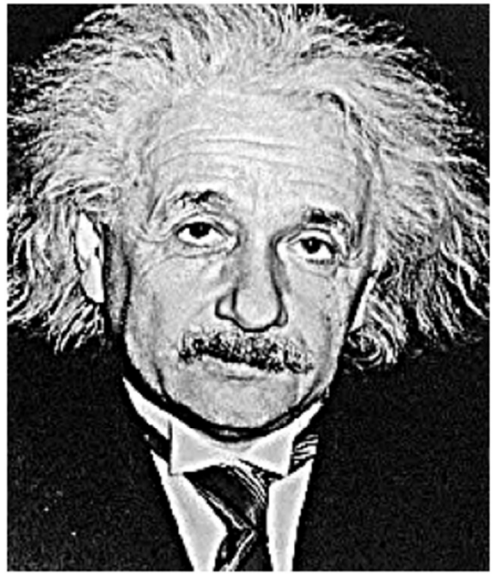
Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias



Unsharp Masking



As we discussed last time, the image eventually becomes saturated.
When the enhancement is too strong, it's as if we start introducing things that aren't there.

What happens if we recursively apply *unsharp masking*?

Imagen Original

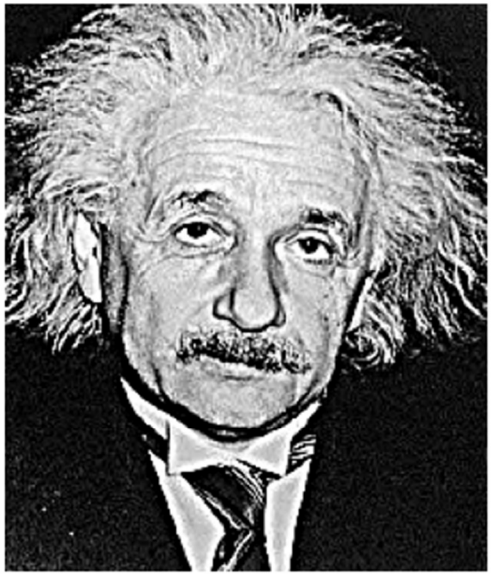
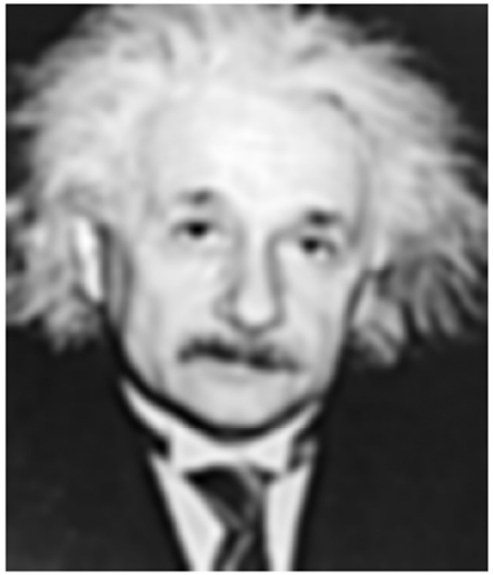
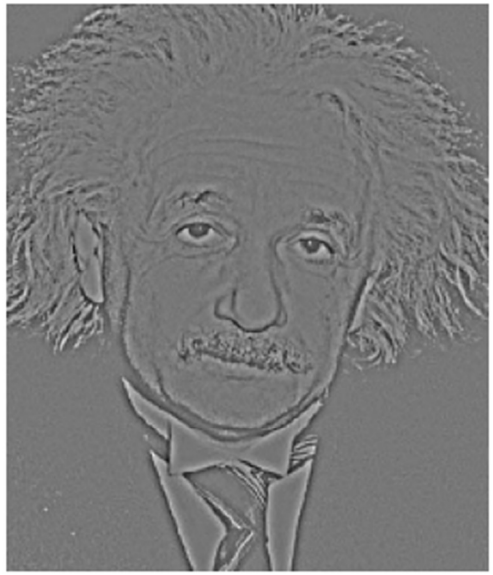


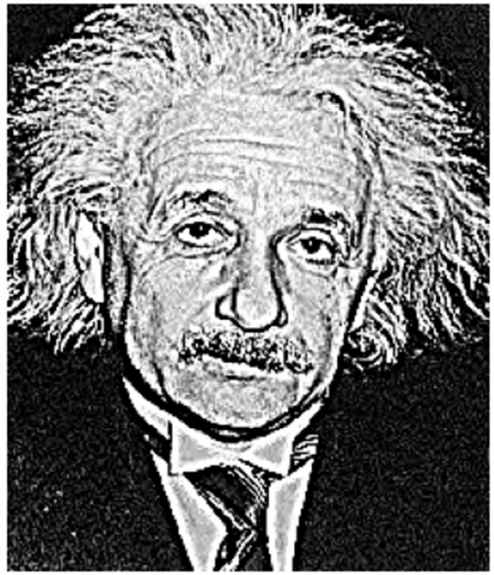
Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias



Unsharp Masking



As we discussed last time, the image eventually becomes saturated.
When the enhancement is too strong, it's as if we start introducing things that aren't there.

What happens if we recursively apply *unsharp masking*?

Imagen Original

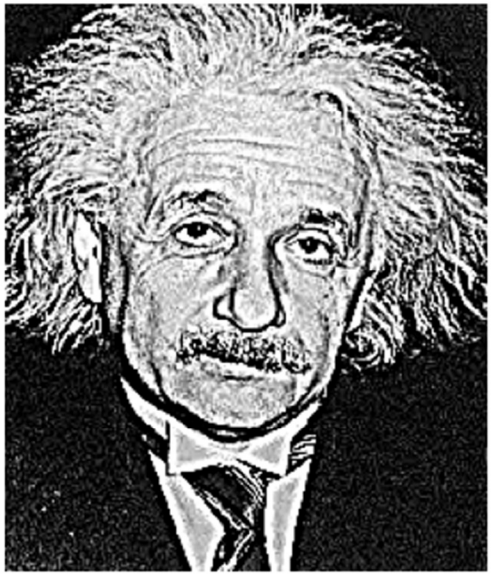
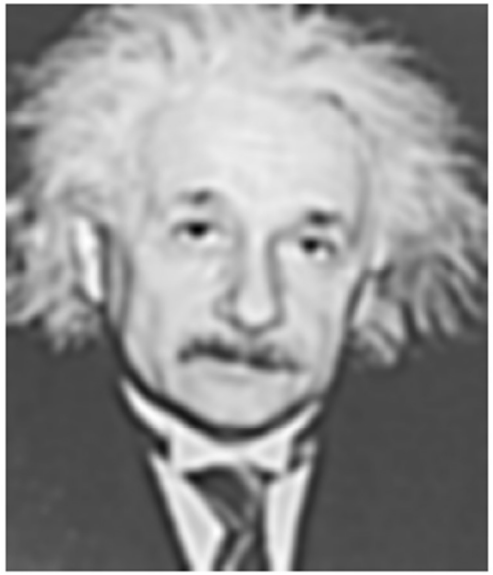
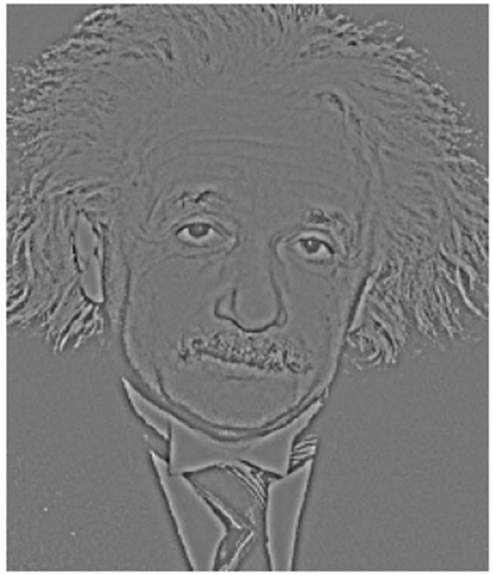


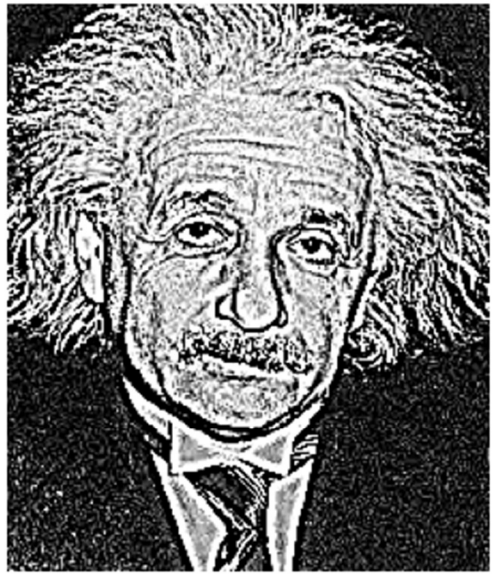
Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias



Unsharp Masking



As we discussed last time, the image eventually becomes saturated.
When the enhancement is too strong, it's as if we start introducing things that aren't there.

What happens if we recursively apply *unsharp masking*?

Imagen Original

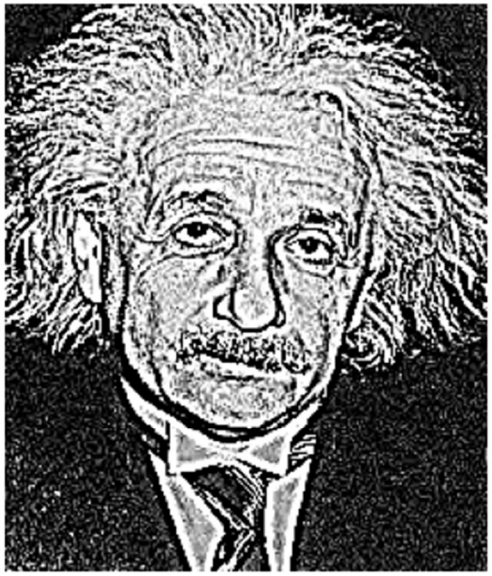
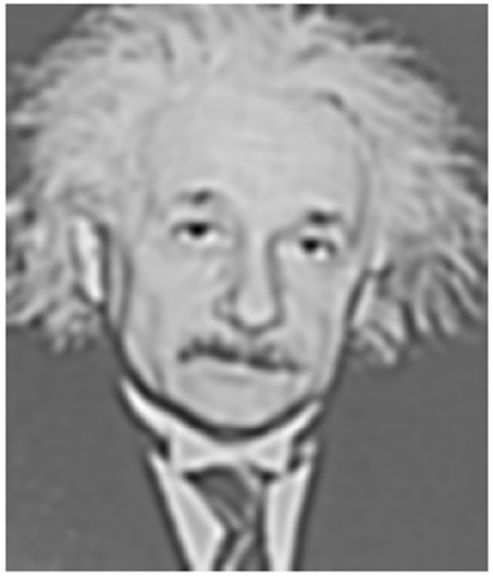
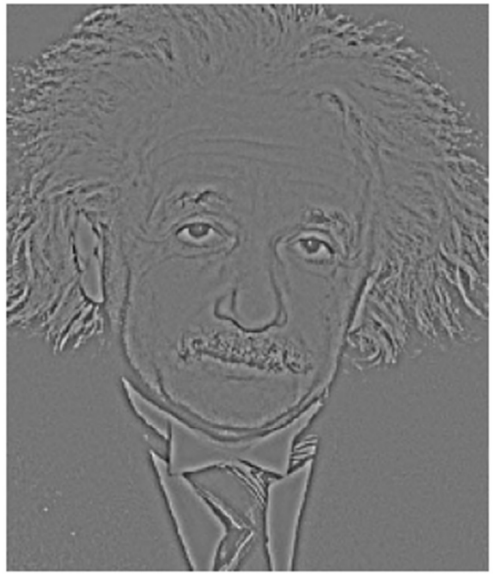


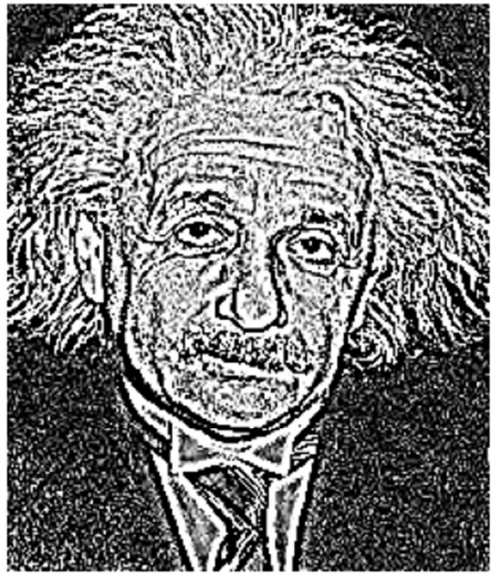
Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias



Unsharp Masking



As we discussed last time, the image eventually becomes saturated.
When the enhancement is too strong, it's as if we start introducing things that aren't there.

What happens if we recursively apply *unsharp masking*?

Imagen Original

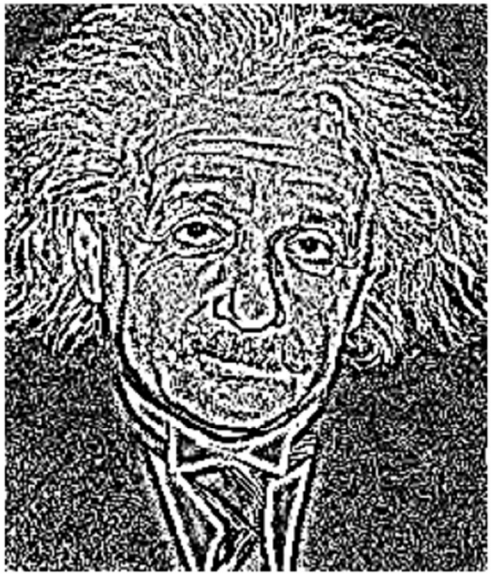
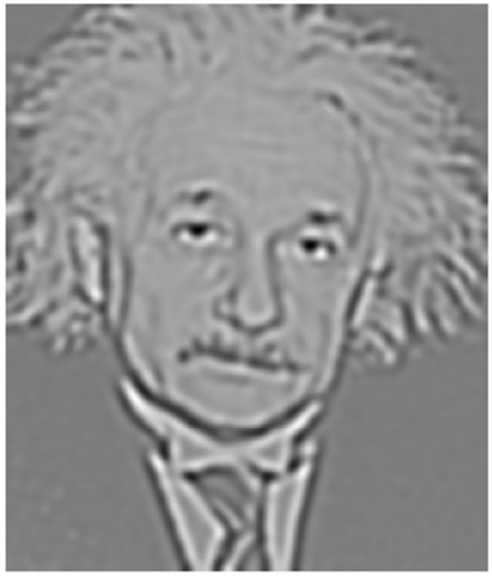
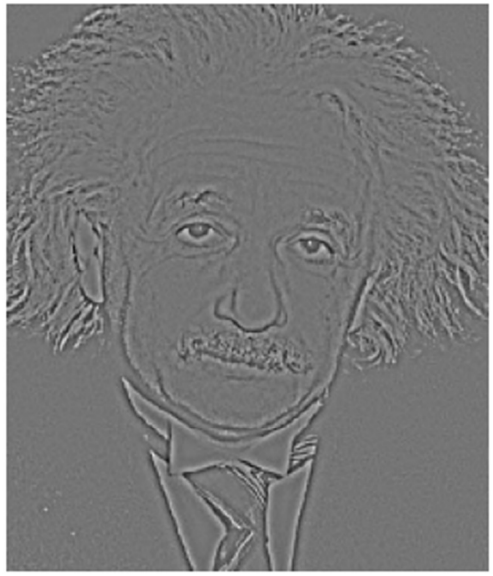


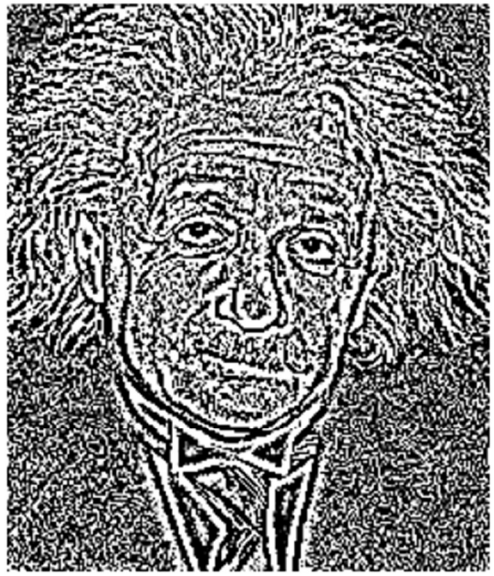
Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias



Unsharp Masking



As we discussed last time, the image eventually becomes saturated.
When the enhancement is too strong, it's as if we start introducing things that aren't there.

What happens if we recursively apply *unsharp masking*?

Imagen Original

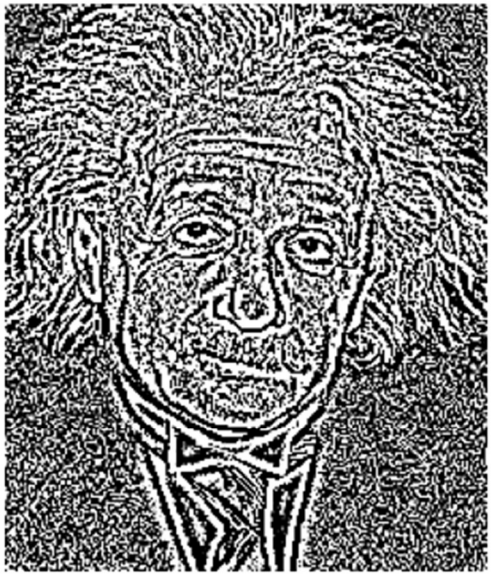
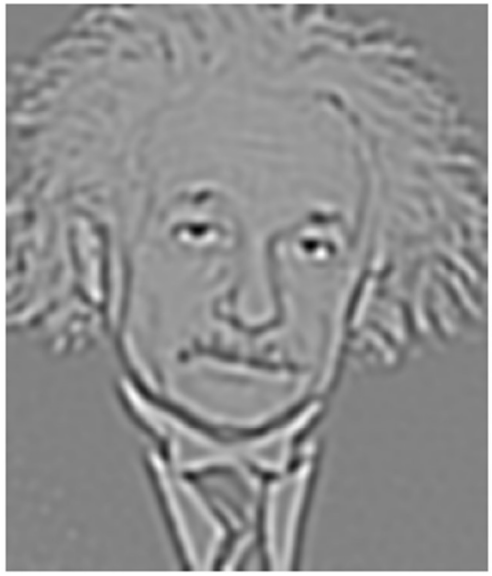
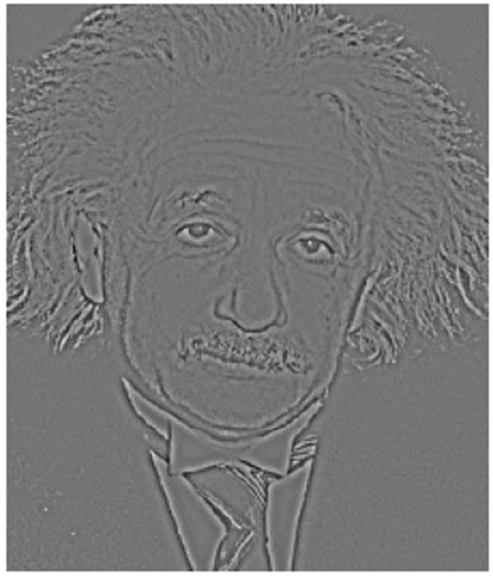


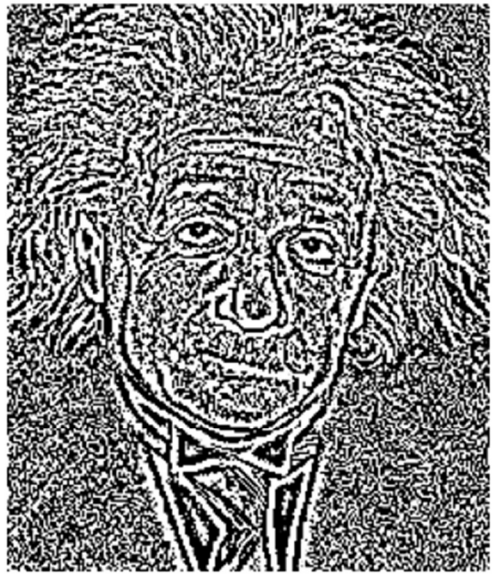
Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias



Unsharp Masking



As we discussed last time, the image eventually becomes saturated.
When the enhancement is too strong, it's as if we start introducing things that aren't there.

What happens if we recursively apply *unsharp masking*?

100 runs later…..

As we discussed last time, the image eventually becomes saturated.
When the enhancement is too strong, it's as if we start introducing things that aren't there.

What happens if we recursively apply *unsharp masking*?

Imagen Original

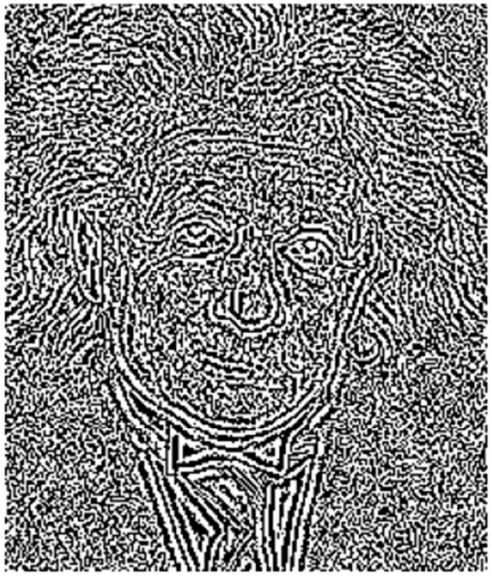


Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias

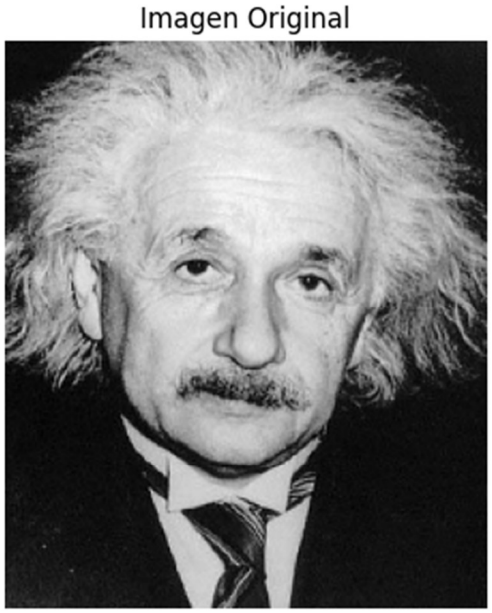


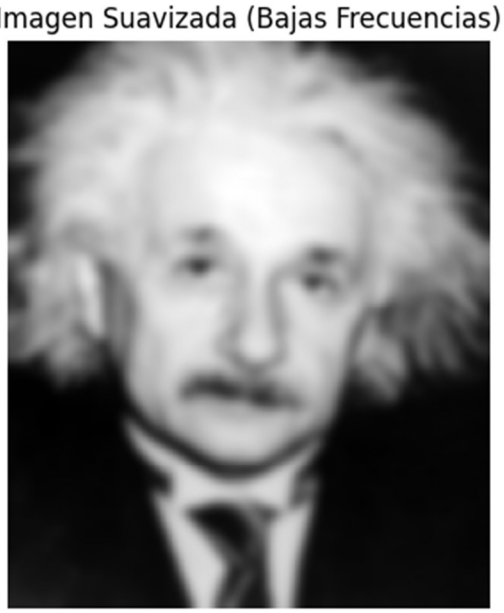
Unsharp Masking

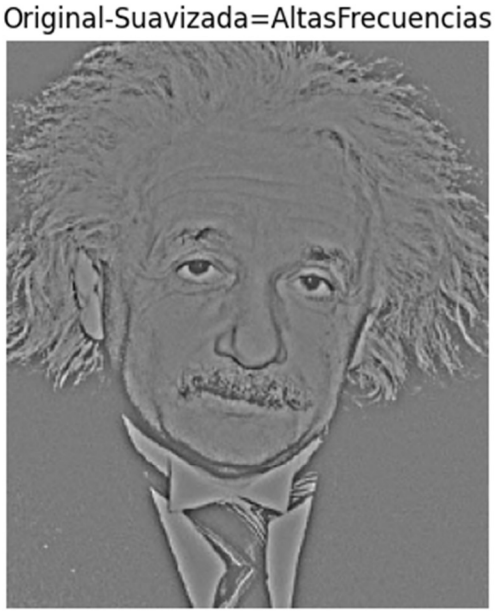


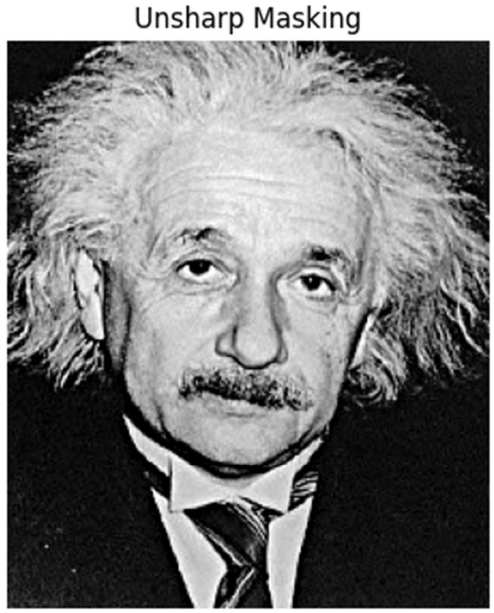
As we discussed last time, the image eventually becomes saturated.
When the enhancement is too strong, it's as if we start introducing things that aren't there.

Increasing α has a similar effect





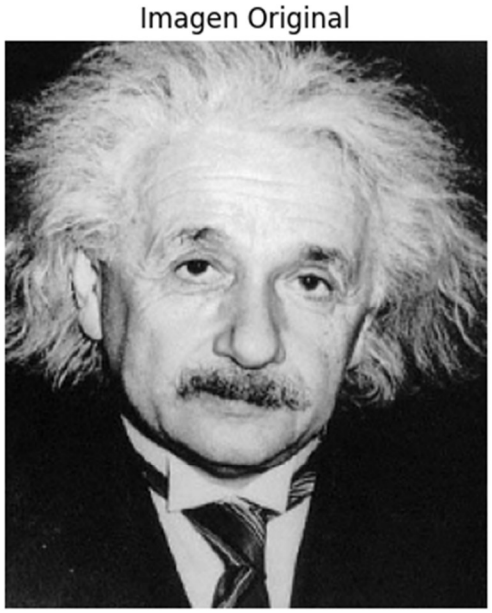


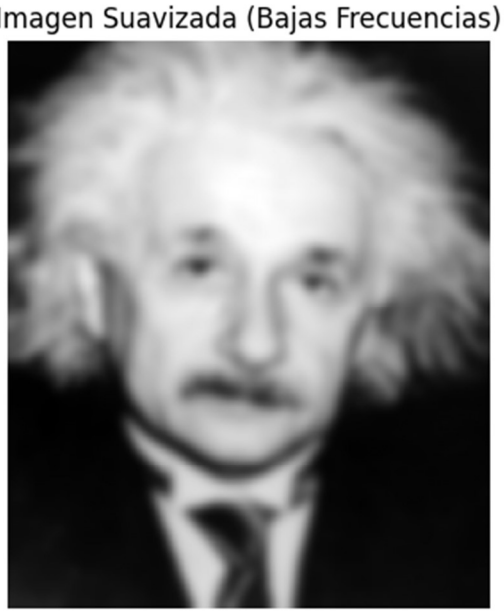


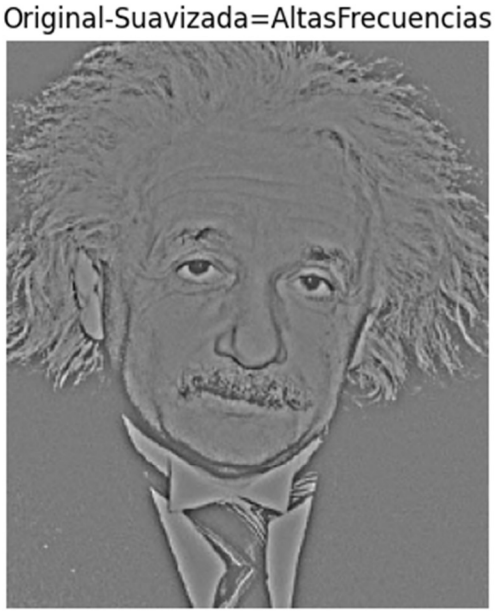
$$F + \alpha\left(F - F * H\right)$$

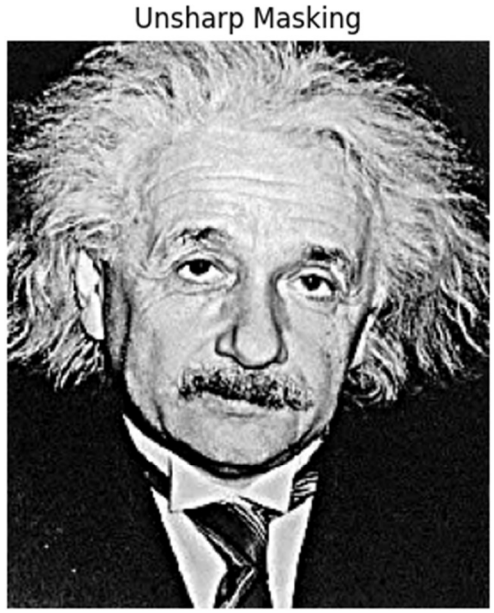
$\alpha = 1.0$

Increasing α has a similar effect





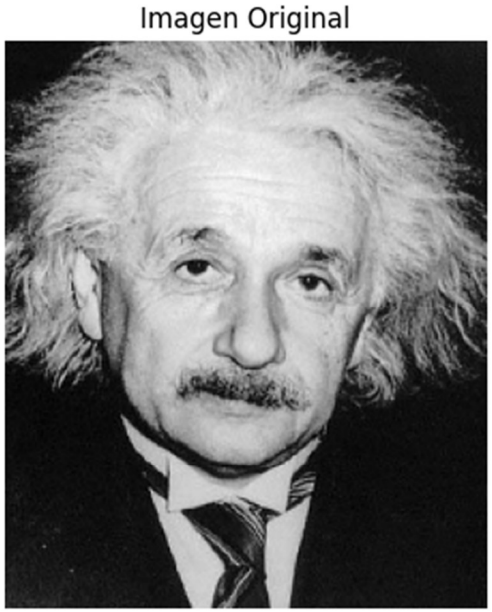


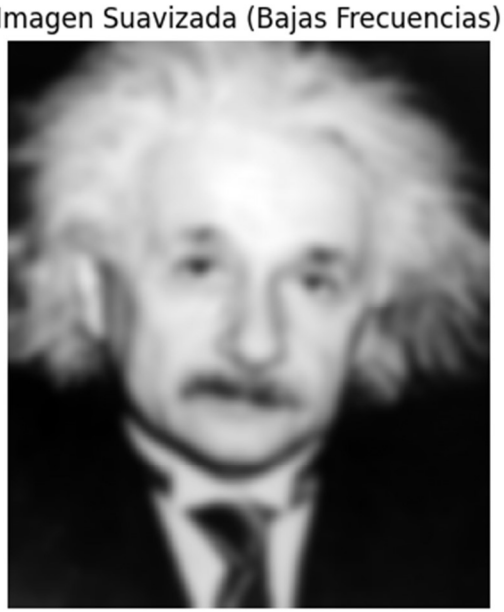


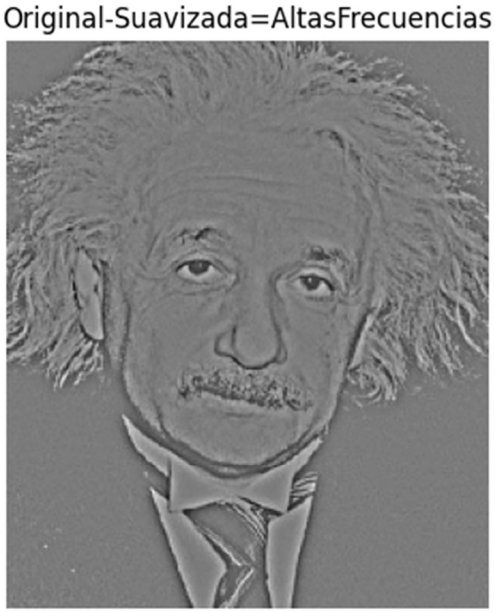
$$F + \alpha(F - F * H)$$

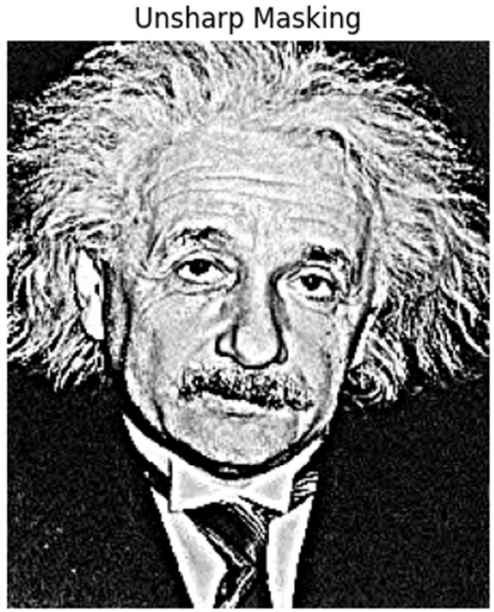
$$\alpha = 2.0$$

Increasing α has a similar effect





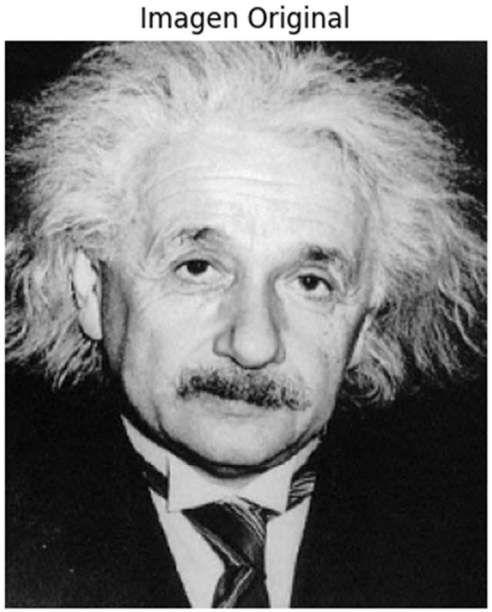


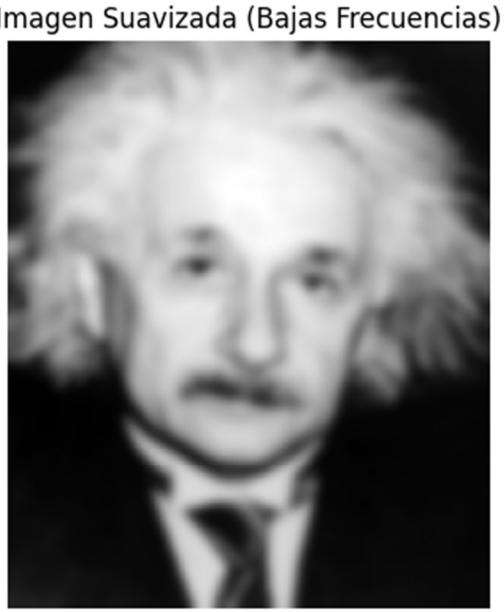


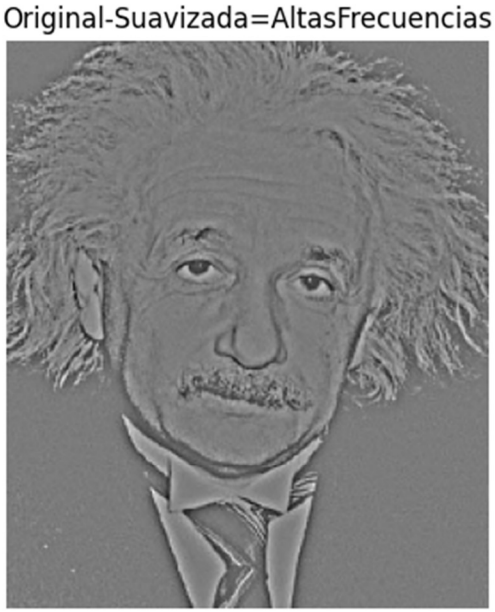
$$F + \alpha(F - F * H)$$

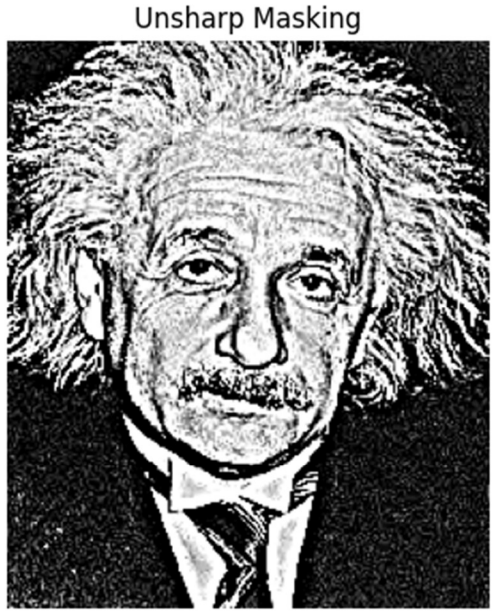
$$\alpha = 5.0$$

Increasing α has a similar effect





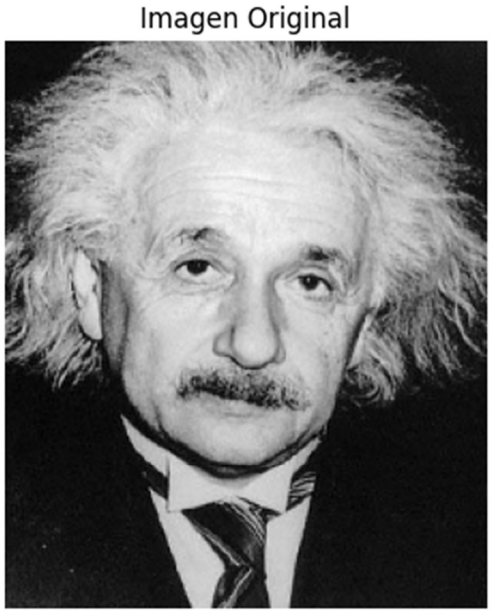


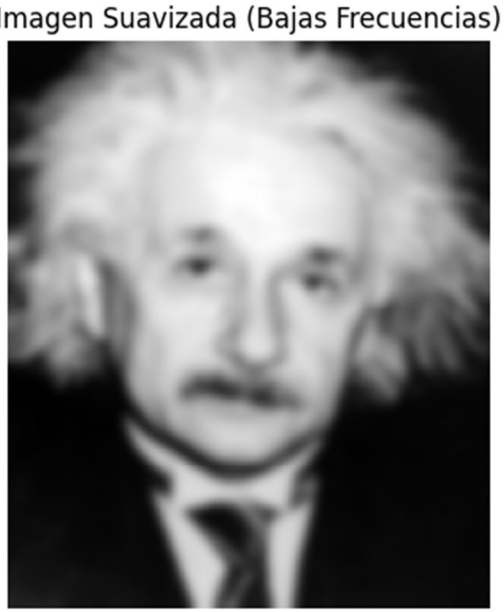


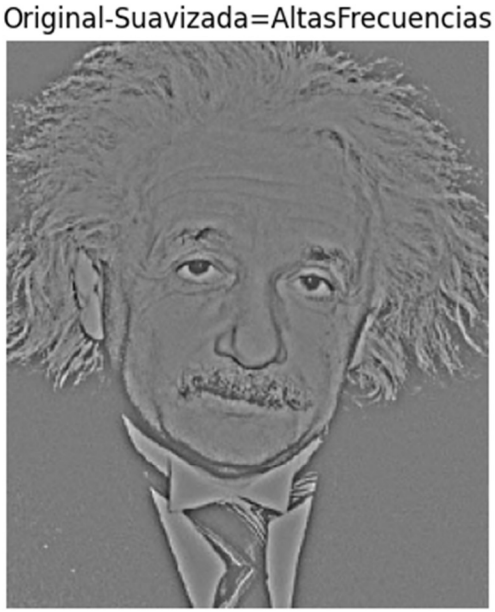
$$F + \alpha(F - F * H)$$

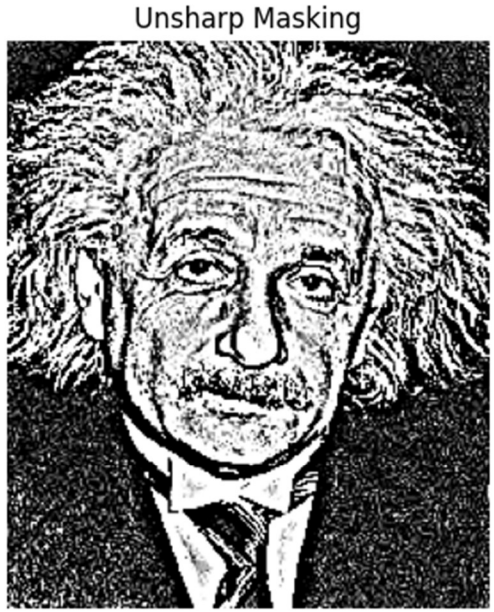
$\alpha = 10.0$

Increasing α has a similar effect





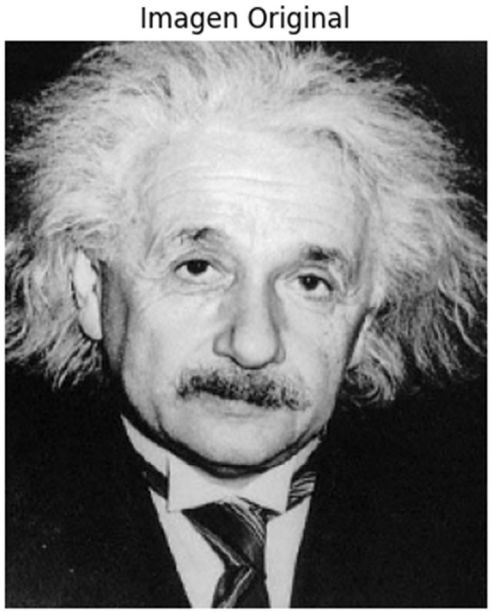


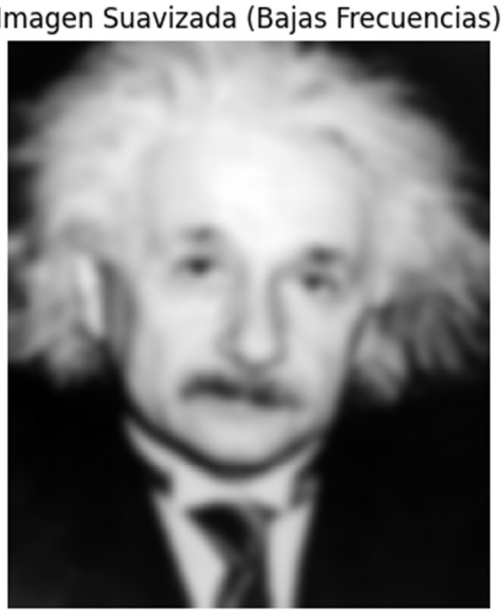


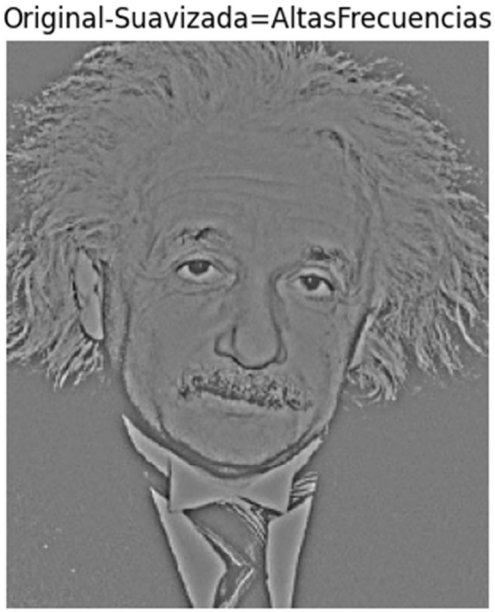
$$F + \alpha(F - F * H)$$

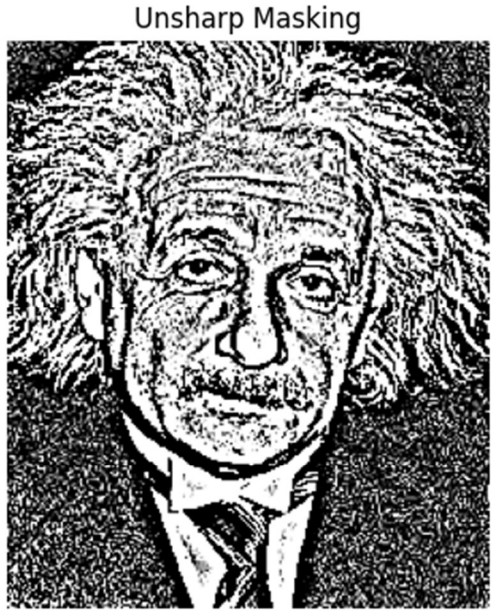
$\alpha = 25.0$

Increasing α has a similar effect





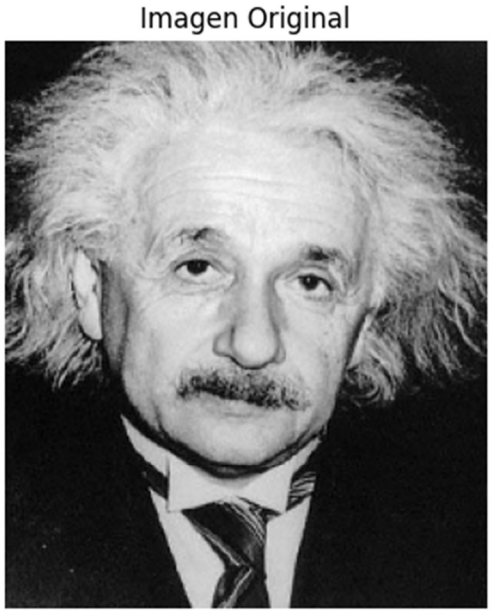


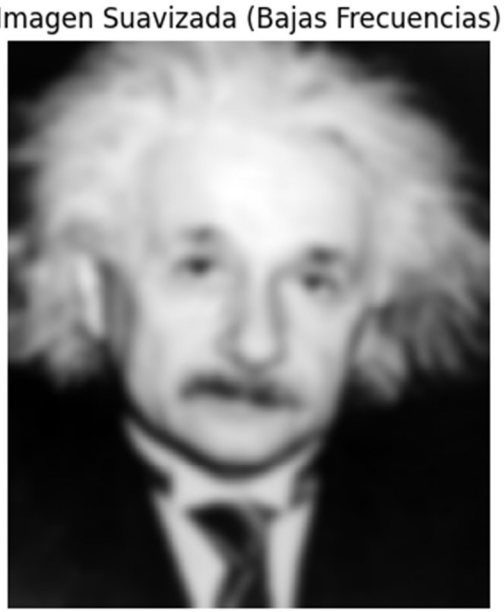


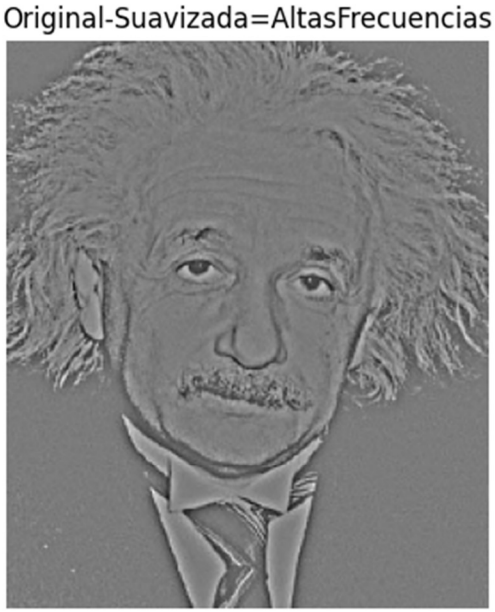
$$F + \alpha(F - F * H)$$

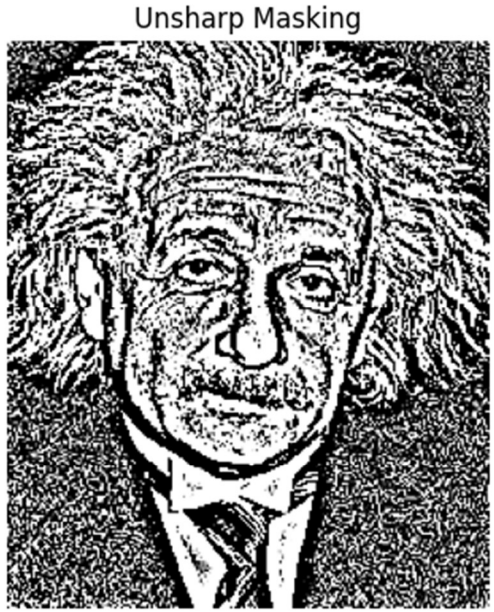
$\alpha = 50.0$

Increasing α has a similar effect





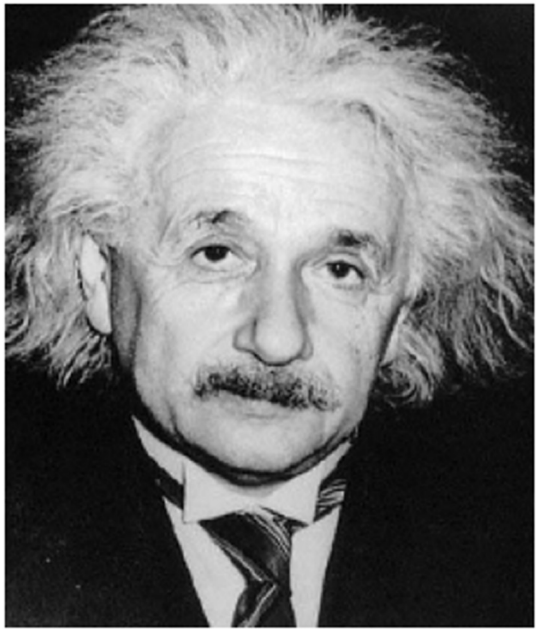


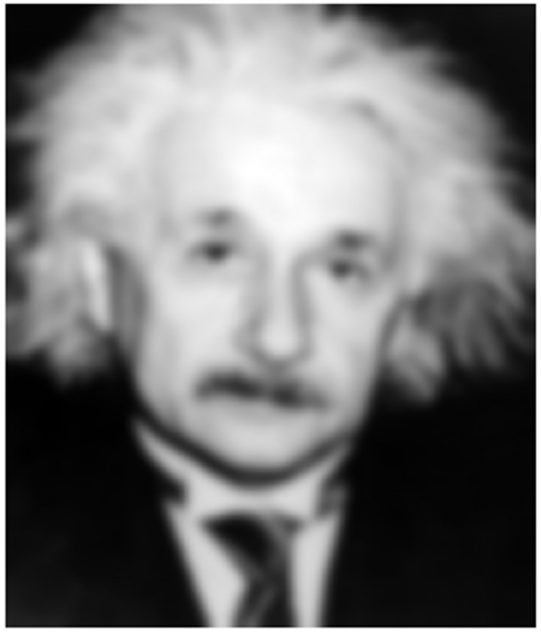


$$F + \alpha\left(F - F * H\right)$$

$$\alpha = 99.0$$

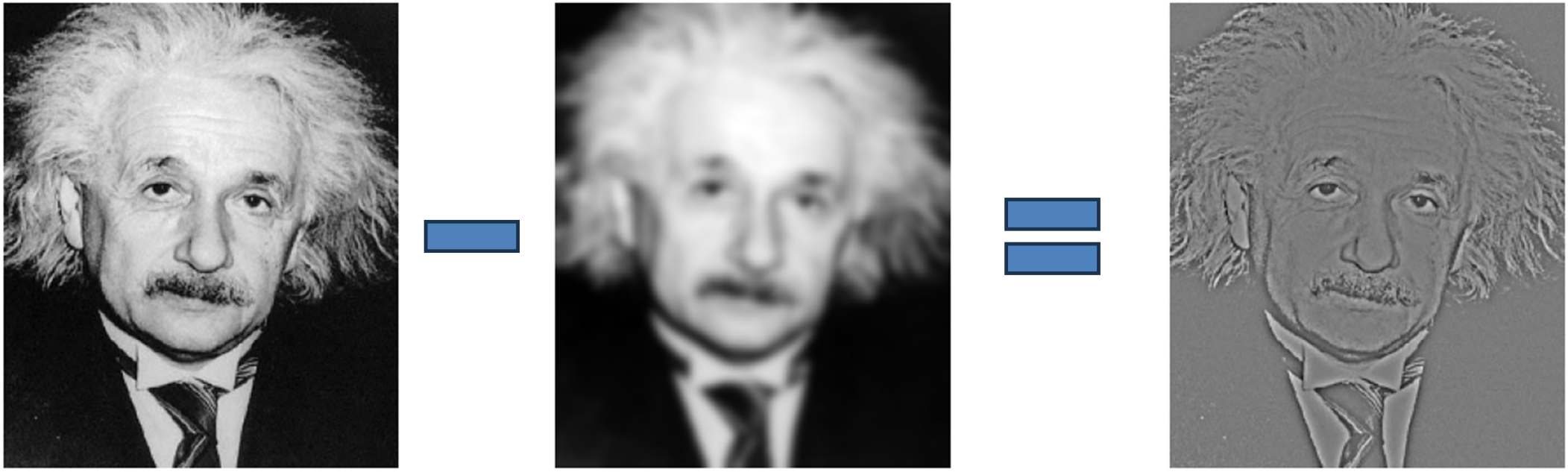
Review questions



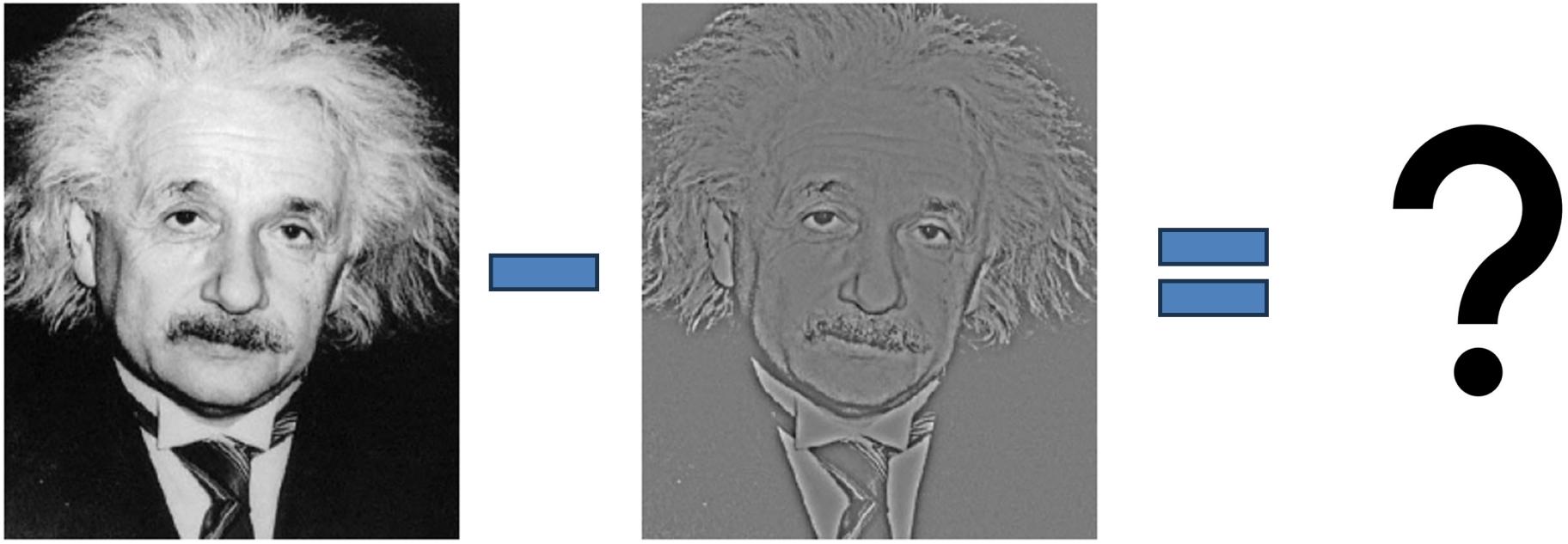


?

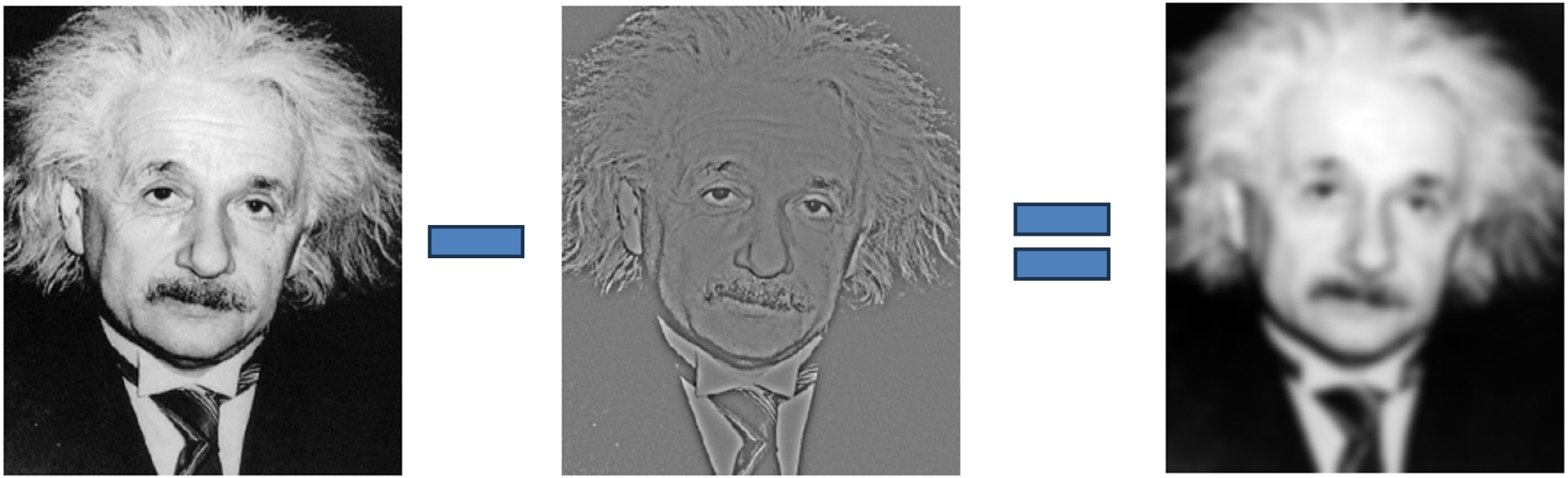
Review questions



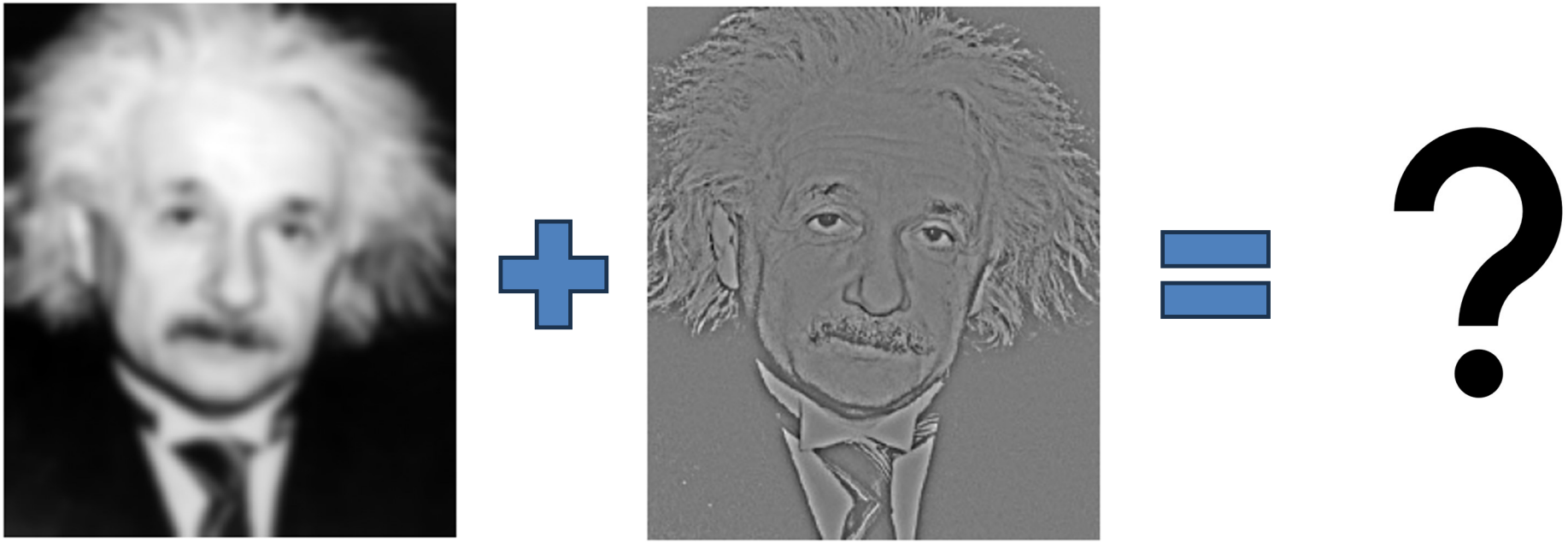
Review questions



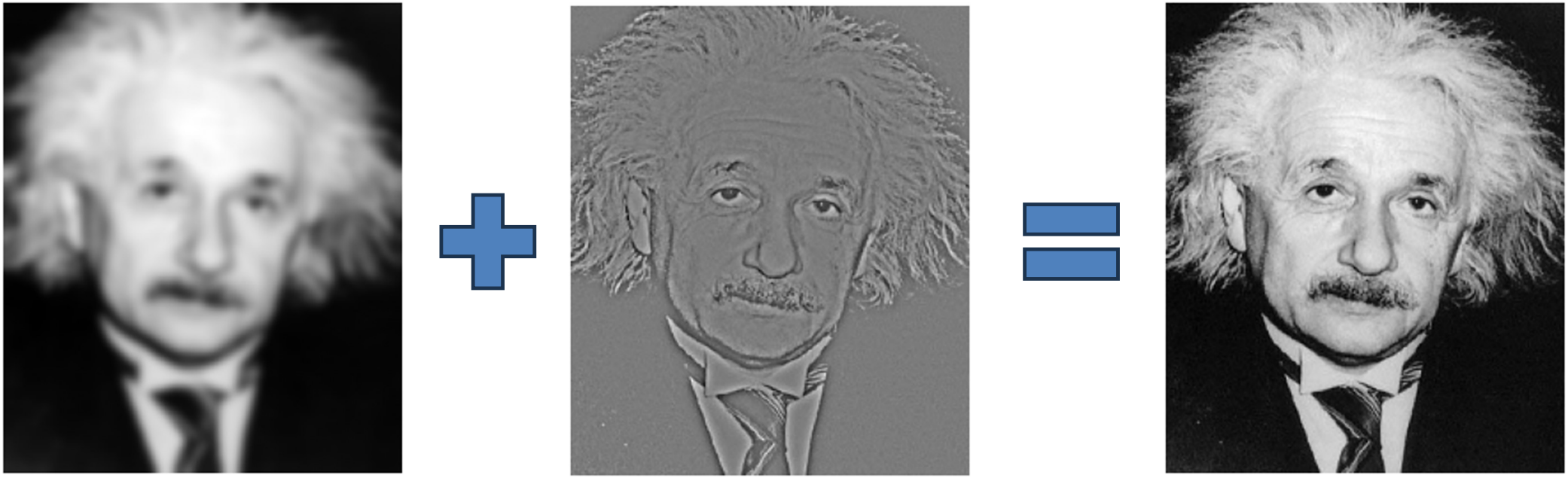
Review questions



Review questions



Review questions



Why is linearity useful?

Superposition lets us understand a complex response from simpler components

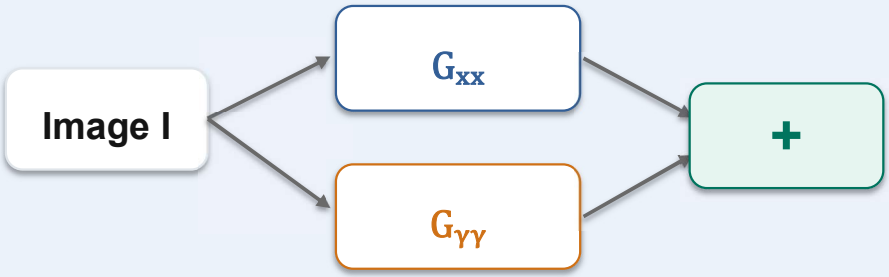
$$T(\alpha x_1 + \beta x_2) = \alpha T(x_1) + \beta T(x_2)$$

Distributivity w.r.t. either argument

$$(I_1 + I_2) * h = I_1 * h + I_2 * h$$
$$I * (h_1 + h_2) = I * h_1 + I * h_2$$

Example: Laplacian of Gaussian

$$I * \nabla^2 G = I * G_{xx} + I * G_{yy}$$



Same result as one convolution with $\nabla^2 G$

Derivatives are simple

A convolution with a fixed kernel is a linear map:

$$y = W x$$

$$\text{Jacobian w.r.t. } x = W$$

In backpropagation, gradients with respect to the learnable weights have a simple form.

Linearity makes derivatives easy to compute and analyze.

Important: "easy derivative" does NOT mean gradients can never vanish or explode.

Is convolution equivariant to rotation and scaling?

First, what does “equivariance” mean?

```
convolution(rotation(Img)) = rotation(convolution(Img))

convolution(scaling(Img)) = scaling(convolution(Img))
```

Is convolution equivariant to translation?

Original Image



Shifted Image



1º Convolution 2º Shift

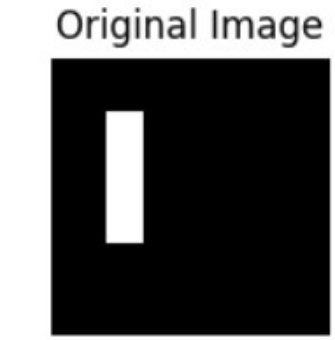


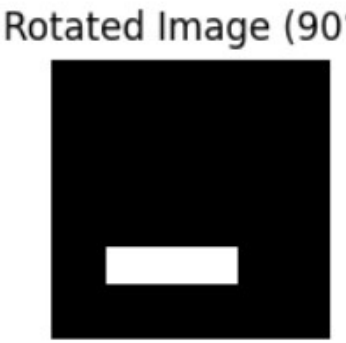
1º Shift 2º Convolution

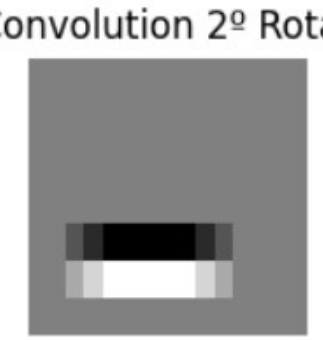


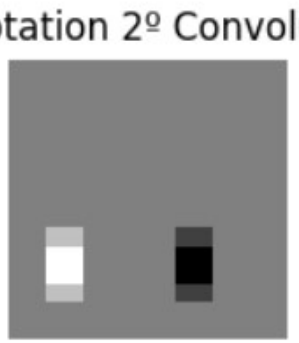
$$\begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}$$

Is convolution equivariant to rotation?









$$\begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}$$

Is convolution equivariant to scaling?

Original Image (15, 15)



Scaled Image (300, 300)



1º Convolution 2º Scaling (300, 300)



1º Scaling 2º Convolution (300, 300)



$$\begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}$$

Is correlation equivalent to convolution if we rotate its output?



$$rotate180(Correlation(I,K)) == Convolution(I,K)$$????

Is correlation equivalent to convolution if we rotate its output?

Input Image



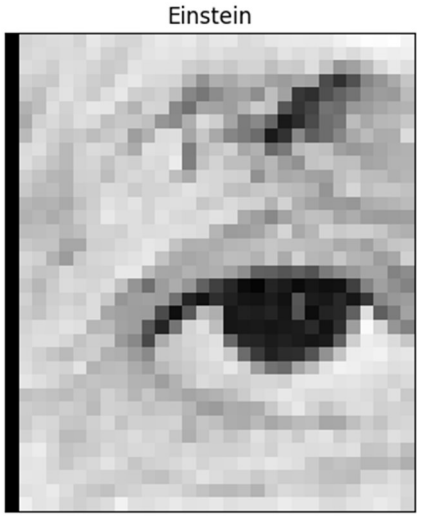
Kernel

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

$Correlation(I, K)$



$Convolution(I, K)$



$rotate180(Correlation(I, K))$



$$\boldsymbol{rotate180(Correlation(I, K)) \neq Convolution(I, K)}$$

Linear filters

- Can thresholding be implemented with a linear filter?

$$g(m,n) = \begin{cases} 255, & f(m,n) > A \\ 0 & otherwise \end{cases}$$

Can thresholding be implemented with a linear filter?

- Can thresholding be implemented with a linear filter?

$$g(m,n) = \begin{cases} 255, & f(m,n) > A \\ 0 & otherwise \end{cases}$$

Thresholding is a non-linear ("if-else") operation!

`L(af+bg) != aL(f)+bL(g)`



Can thresholding be implemented with a linear filter?

- Can thresholding be implemented with a linear filter?

Let us set the threshold to A = 120. Consider a pixel:

$$x = 70 \rightarrow T(70) = 0$$

Now multiply it by 2:

$$T(2 * 70) = T(140) = 255$$

If T were linear, we should have:

$$T(2 * 70) = 2 * T(70) = 0$$

But:

$$255 \; != \; 0$$

Therefore, thresholding is not a linear operation.

Can thresholding be implemented with a linear filter?

- Can thresholding be implemented with a linear filter?

$$g(m,n) = \begin{cases} 255, & f(m,n) > A \\ 0 & otherwise \end{cases}$$

$a = 2.0$
$Threshold = 120$



Can thresholding be implemented with a linear filter?

- Can thresholding be implemented with a linear filter?

$$g(m,n) = \begin{cases} 255, & f(m,n) > A \\ 0 & otherwise \end{cases}$$

Thresholding is a non-linear ("if-else") operation!

- There's no proportional change in the output wrt the input:
  - In a linear filter, if you change a pixel's intensity slightly, the output changes proportionally.
  - However, here, small changes in the pixel intensity near the threshold can result in large, discontinuous jumps in the output.
- The output is not a weighted sum of pixel intensities

Linear filters

- Which one of the following 3x3 filters returns a **positive value if the average value of the 4-adjacent neighbors is less than the center**, and a negative value (or zero) otherwise?

(a)

0	¼	0
¼	-1	¼
0	¼	0

(b)

-1	-2	-1
0	0	0
1	2	2

(c)

0	-1	0
-1	4	-1
0	-1	0

Linear filters

- Which one of the following 3x3 filters returns a **positive value if the average value of the 4-adjacent neighbors is less than the center**, and a negative value (or zero) otherwise?

